

THỰC HÀNH THIẾT KẾ HỆ THỐNG SOC - LỚP **CE433.Q21.1**

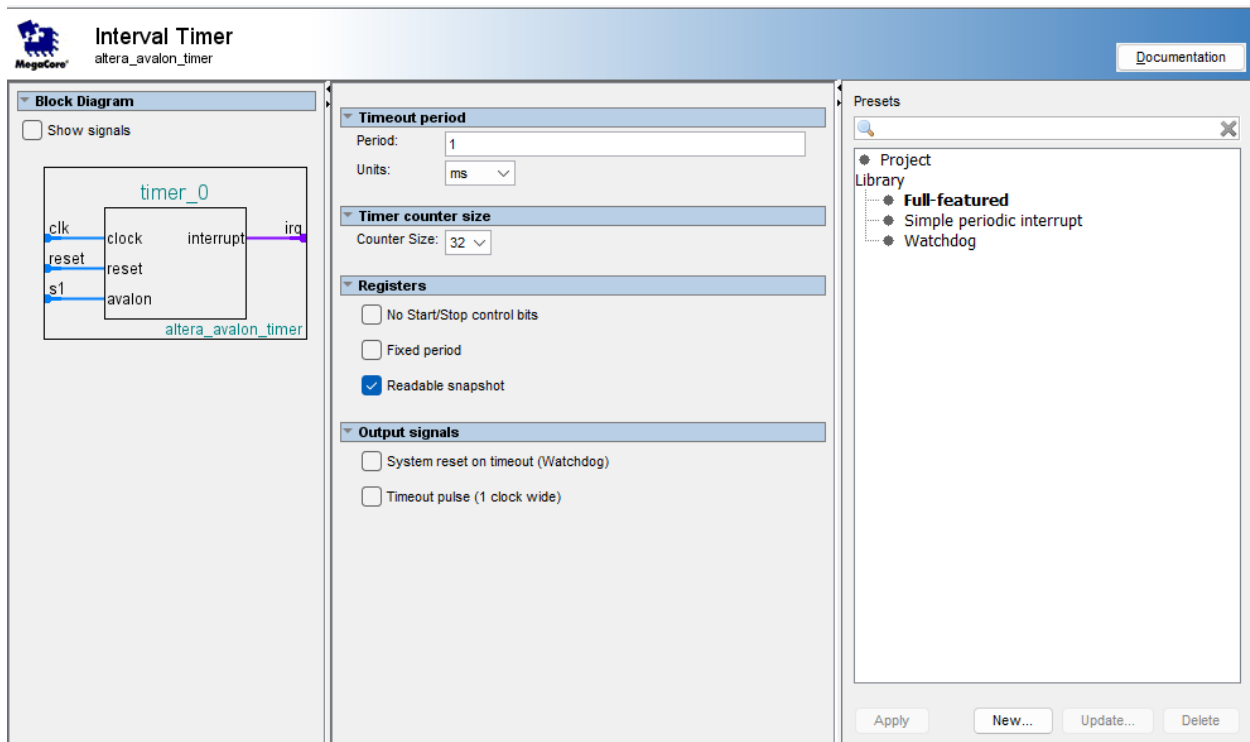
BÀI THỰC HÀNH 4

Giảng viên hướng dẫn	Trương Văn Cường		ĐIỂM
Sinh viên thực hiện	Vũ Đại Dương	23520359	

Bài 1. Xây dựng mô đun hệ thống SoC dùng Timer, SW, HEX tạo thành đồng hồ.
Cách tùy chỉnh đồng hồ cho sinh viên tự quy định.

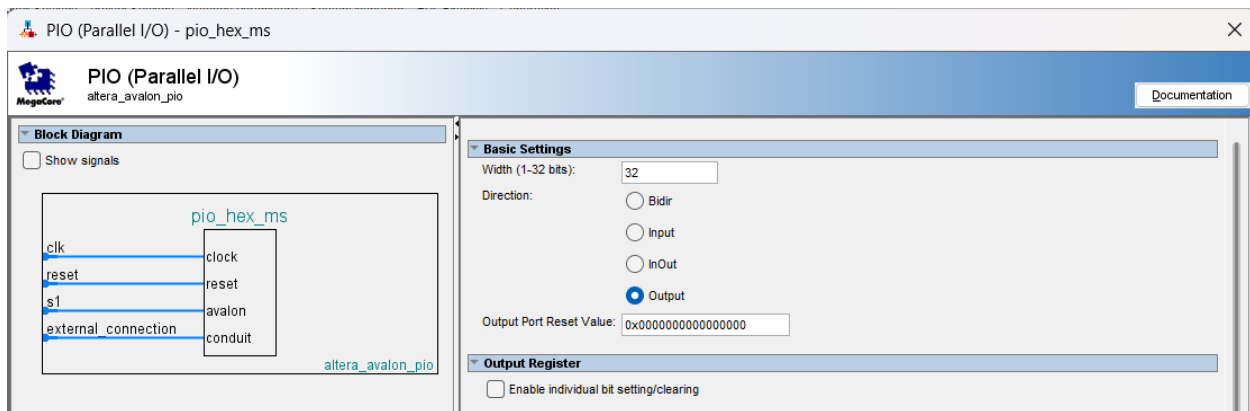
Bài làm:

1. Thiết kế hệ thống Qsys:
 - Thêm timer cho hệ thống:

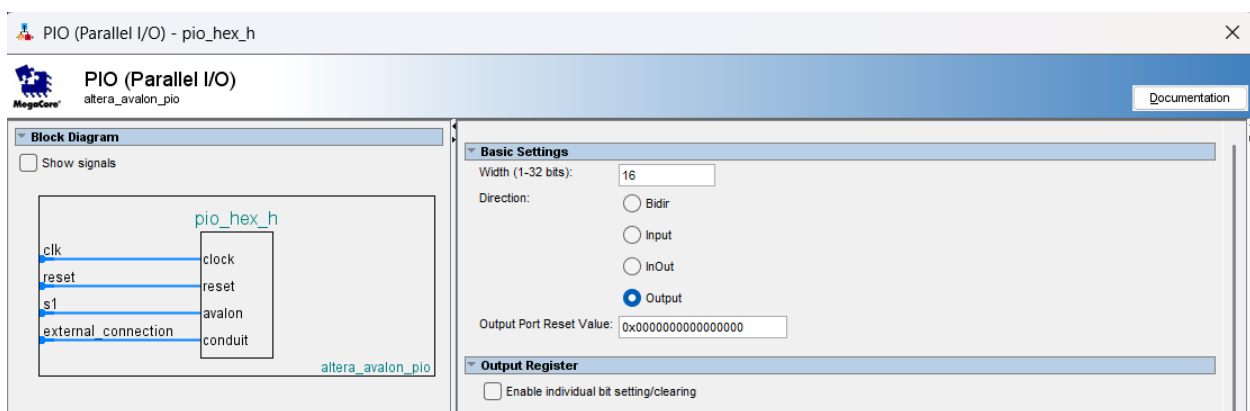


Hình 1 Set up Timer

- Thêm các pio để hiển thị LED 7 đoạn:

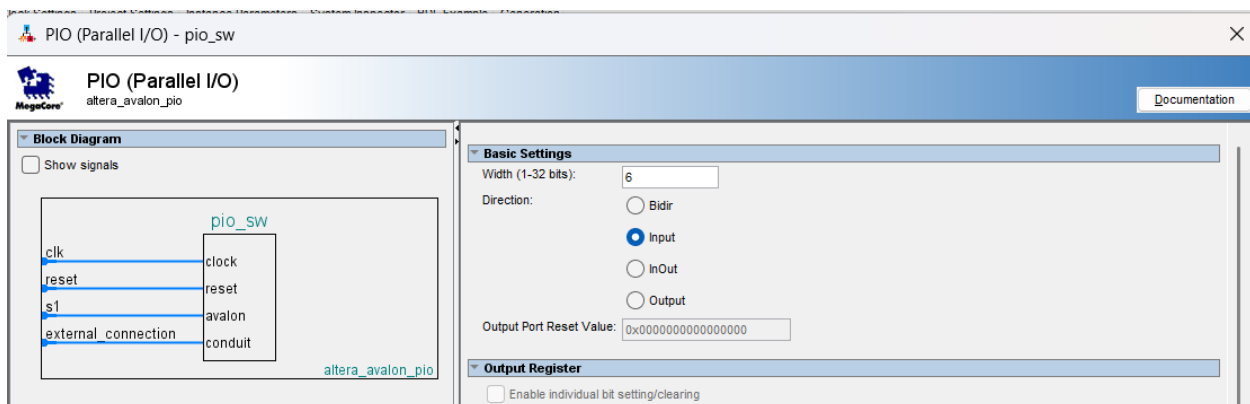


Hình 2 Set up pio_hex_ms



Hình 3 Set up pio_hex_h

- pio_hex_ms: dùng để hiển thị giá trị phút và giây (chục và đơn vị) ra 4 LED 7 đoạn → cần 28 bit nhưng làm tròn lên 32 bit để truyền đầy đủ hex
- pio_hex_h: dùng để hiển thị giá trị giờ (chục và đơn vị) ra 2 LED 7 đoạn → cần 14 bit nhưng làm tròn lên 16 bit để truyền đầy đủ hex
- Đặt direction là Output để hiển thị kết quả.
- Thêm pio điều khiển switch để thực hiện điều chỉnh thời gian:



Hình 4 Set up pio_sw

- Sử dụng 6 bit là 6 switch để điều chỉnh thời gian, với 3 switch tăng giờ/phút/giây 1 đơn vị và 3 switch để reset giờ/phút/giây.
- Toàn bộ hệ thống:

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
✓		clock	Clock Source						
		clk_in	Clock Input	clk					
		clk_in_reset	Reset Input	reset					
		clk	Clock Output	Double-click to export	clock				
		clk_reset	Reset Output	Double-click to export					
✓		nios2_qsys_0	Nios II Processor						
		clk	Clock Input	Double-click to export	clock				
		reset_n	Reset Input	Double-click to export	[clk]				
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]			IRQ 0	
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
		jtag_debug_module_re	Reset Output	Double-click to export	[clk]			IRQ 31	
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_0800	0x0002_0fff		
		custom_instruction_m...	Custom Instruction Master	Double-click to export	[clk]				
✓		onchip_memory2_0	On-Chip Memory (RAM or ROM)						
		clk1	Clock Input	Double-click to export	clock				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	# 0x0001_0000	0x0001_9fff		
		reset1	Reset Input	Double-click to export	[clk1]				
✓		jtag_uart_0	JTAG UART						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1058	0x0002_105f		
✓		timer_0	Interval Timer						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1000	0x0002_101f		
✓		pio_sw	PIO (Parallel IO)						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1040	0x0002_104f		
		external_connection	Conduit	pio_sw_export					
✓		pio_hex_ms	PIO (Parallel IO)						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1030	0x0002_103f		
		external_connection	Conduit	pio_hex_ms_export					
✓		pio_hex_h	PIO (Parallel IO)						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1020	0x0002_102f		
		external_connection	Conduit	pio_hex_h_export					

Hình 5 Hệ thống trên Qsys

2. Thiết kế top module

```
module Bai7(
    input CLOCK_50,
    input [0:0] KEY,
    input [5:0] SW,
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5
);
```

```
wire [31:0] hex_ms_wire;
wire [15:0] hex_h_wire;
```

```
assign HEX0 = ~hex_ms_wire[6:0];
assign HEX1 = ~hex_ms_wire[14:8];
assign HEX2 = ~hex_ms_wire[22:16];
assign HEX3 = ~hex_ms_wire[30:24];
```

```
assign HEX4 = ~hex_h_wire[6:0];
assign HEX5 = ~hex_h_wire[14:8];
```

```
system Nios_system(
    .clk_clk (CLOCK_50),
```

```

.reset_reset_n      (KEY[0]),
.pio_sw_export_export      (SW),
.pio_hex_ms_export_export (hex_ms_wire),
.pio_hex_h_export_export   (hex_h_wire)
);
endmodule

```

Bảng 1 Code verilog Bai7.v

- HEX0: hiển thị đơn vị giây
- HEX1: hiển thị chục giây
- HEX2: hiển thị đơn vị phút
- HEX3: hiển thị chục phút
- HEX4: hiển thị đơn vị giờ
- HEX5: hiển thị chục giờ

3. Hiện thực hóa bằng Eclipse

```

#include <stdio.h>
#include <unistd.h>
#include "system.h"
#include "altera_avalon_pio_regs.h"
#include "altera_avalon_timer_regs.h"
#include "sys/alt_irq.h"

const unsigned char hex_table[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D,
0x7D, 0x07, 0x7F, 0x6F};

volatile int s = 0, m = 0, h = 0;
volatile int ms_count = 0;
volatile int timer_running = 1;

void timer_isr(void* context) {
    IOWR_ALTERA_AVALON_TIMER_STATUS(TIMER_0_BASE, 0);

    if (timer_running) {
        ms_count++;
        if (ms_count >= 1000) {
            ms_count = 0;
            s++;

            if (s >= 60) {
                s = 0; m++;
                if (m >= 60) {
                    m = 0; h++;
                    if (h >= 24) h = 0;
                }
            }
        }
    }
}

void init_timer() {

```

```

IOWR_ALTERA_AVALON_TIMER_CONTROL(TIMER_0_BASE, 0x0008);
IOWR_ALTERA_AVALON_TIMER_STATUS(TIMER_0_BASE, 0);
alt_ic_isr_register(TIMER_0_IRQ_INTERRUPT_CONTROLLER_ID, TIMER_0_IRQ,
timer_isr, NULL, 0);
IOWR_ALTERA_AVALON_TIMER_CONTROL(TIMER_0_BASE, 0x0007);
}

int main() {
    init_timer();
    int last_sw = 0;

    while (1) {
        int sw_val = IORD_ALTERA_AVALON_PIO_DATA(PIO_SW_BASE);

        int adj_s = (sw_val >> 0) & 1;
        int adj_m = (sw_val >> 1) & 1;
        int adj_h = (sw_val >> 2) & 1;
        int res_s = (sw_val >> 3) & 1;
        int res_m = (sw_val >> 4) & 1;
        int res_h = (sw_val >> 5) & 1;

        if (adj_s || adj_m || adj_h) timer_running = 0;
        else timer_running = 1;

        if (res_s) { s = 0; ms_count = 0; }
        if (res_m) { m = 0; ms_count = 0; }
        if (res_h) { h = 0; ms_count = 0; }

        if (adj_s && !((last_sw >> 0) & 1)) { s++; ms_count = 0;
if(s>=60) s=0; }
        if (adj_m && !((last_sw >> 1) & 1)) { m++; ms_count = 0;
if(m>=60) m=0; }
        if (adj_h && !((last_sw >> 2) & 1)) { h++; ms_count = 0;
if(h>=24) h=0; }

        last_sw = sw_val;

        unsigned int hex_ms_data = 0;
        unsigned int hex_h_data = 0;

        hex_ms_data |= (hex_table[s % 10] << 0);
        hex_ms_data |= (hex_table[s / 10] << 8);
        hex_ms_data |= (hex_table[m % 10] << 16);
        hex_ms_data |= (hex_table[m / 10] << 24);

        hex_h_data |= (hex_table[h % 10] << 0);
        hex_h_data |= (hex_table[h / 10] << 8);

        IOWR_ALTERA_AVALON_PIO_DATA(PIO_HEX_MS_BASE, hex_ms_data);
        IOWR_ALTERA_AVALON_PIO_DATA(PIO_HEX_H_BASE, hex_h_data);

        usleep(10000);
    }
    return 0;
}

```

Bảng 2 Code C để hiển thị kết quả lên DE2

- Hiện thực trên kit DE2: [Video lab4](#)